

COMS 6901 · Internet Real-Time Lab · Fall 2026

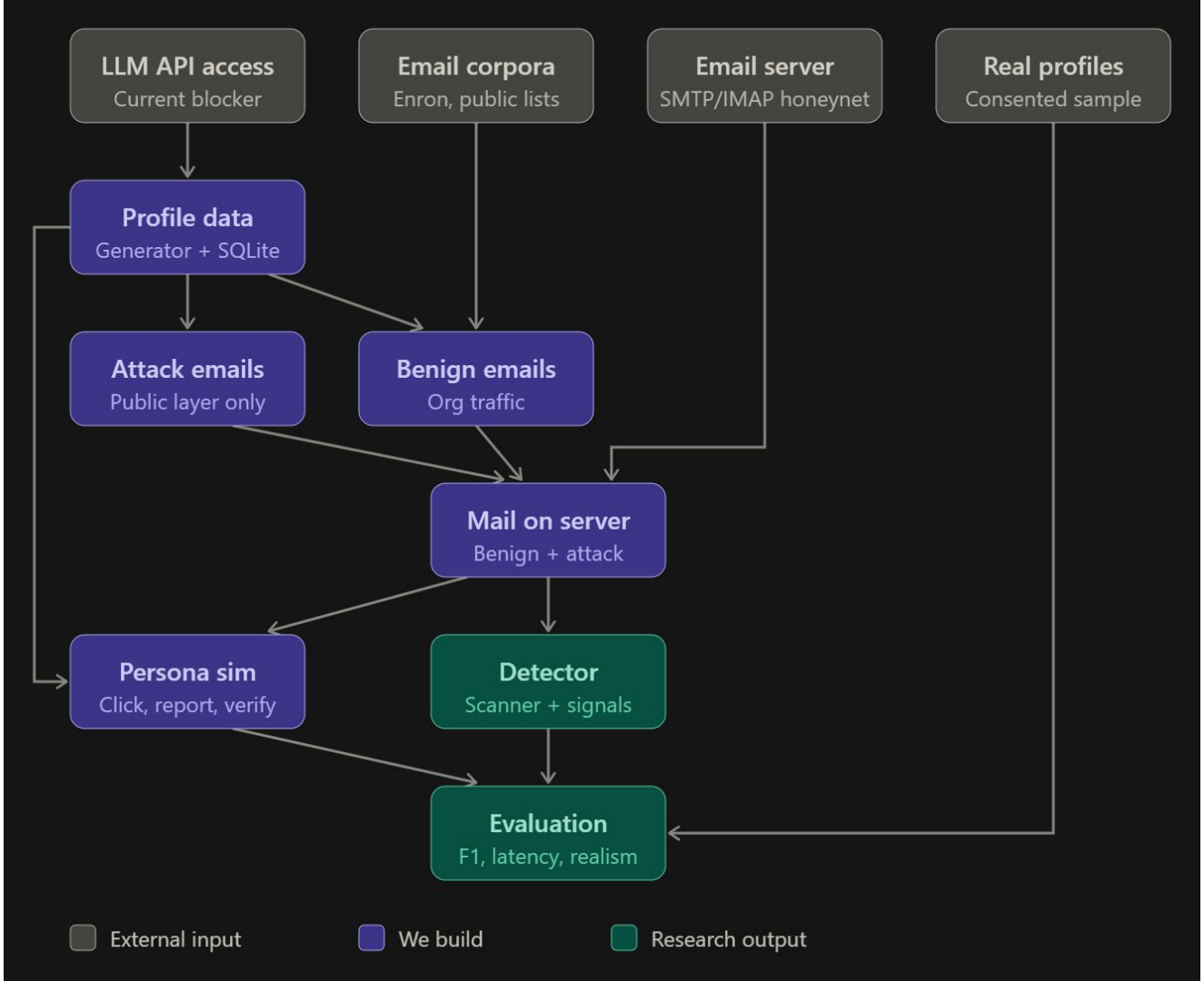
Generative AI for Spear Attack Mitigation

DATE

October 12th, 2026

ADVISORS

Dr. Gaston Ormazabal
Prof. Henning Schulzrinne


LLM API access
Current blocker
Email corpora
Enron, public lists
Email server
SMTP/IMAP honeynet
Real profiles
Consented sample
Profile data
Generator + SQLite
Attack emails
Public layer only
Benign emails
Org traffic
Mail on server
Benign + attack
Persona sim
Click, report, verify
Detector
Scanner + signals
Evaluation
F1, latency, realism
External input
We build
Research output

Email Web Server: SMTP/IMAP Honeynet

Everyone

A decoy mail environment: attackers and our own tests hit a realistic server, and no real person is exposed.

Sender compromised (e.g. IBM) → **Source validated as IBM** → **Spear email delivered** → **Content flagged as malicious**

The gap we close

- Our system can tell a spear email is malicious without first knowing whether it is spam
- If an attacker gets into IBM and sends spear emails, the source is validated as IBM
- Trusting the sender alone leaves this external gap open
- Goal: close this external vulnerability gap

Systems questions

- Latency: how fast can we receive, scan, and deliver?
- Performance and throughput under load
- Integration with Chrome and/or server hardware
- Where scanning should sit in the mail path

Synthetic Employee Profile Generation

Justin & Ayaan

5 Industries: Finance, Healthcare, Tech, Government, Education

Career Stages: Student, Entry Level, Mid Level, High Level, Executive

Company size: Small - lacks dedicated security staff, Medium, Large - more money/data

Prime Targets:

- **Executives and senior leaders**: Often called whaling when aimed at C-suite members, since they have broad access and authority.
- **Finance and accounting staff**: These employees can authorize payments, so they're the usual targets of business email compromise (BEC) schemes, like fake invoices or urgent wire transfer requests.
- **HR and payroll staff**: They hold sensitive employee data (tax forms, SSNs) and can redirect direct deposits.
- **IT administrators and help desk staff**: They have privileged credentials and system access, so compromising one can open the whole network.
- **Executive assistants**: They have access to an executive's inbox, calendar, and accounts, and are often trusted to act on the executive's behalf.
- **Employees with access to valuable data**: This includes engineers or researchers with intellectual property, and legal staff handling confidential matters.
- **Government, military, and defense personnel**: Frequent targets of espionage-focused campaigns, often by state-sponsored groups.
- **Journalists, activists, and political figures**: Targeted for surveillance or to leak information.
- **New hires**: They're less familiar with internal processes and colleagues, so impersonating a manager or IT is more likely to work.

Profiles Organization + Creation:

Justin & Ayaan

```
profiles_core.jsonl      # identity, sector, stage, employer
profiles_public.jsonl    # resume bullets, social presence, footprint level, posts over time, personal website?
profiles_private.jsonl   # projects, colleagues, real details behind resume lines -> provides a baseline for under-knowledge comparison
profiles_behavior.jsonl  # traits and response tendencies
orgs.jsonl # companies, domains, org charts
emails_benign.jsonl      # synthetic internal traffic
emails_attack.jsonl      # generated spear emails
sim_results.jsonl        # ignore/reply/click/verify/report/comply outcomes
```

The database is SQLite, with one table per record type from the schema, all linked by a `profile_id`.

The profiles will be created in batches by different llms to prevent them from being too similar.

Email traffic will be generated using an llm trained on the email dataset.

Profiles will be created in aggregate by company (artificial with real website) with the companies having varying sizes and belonging to the respective fields.

Potential late-stage step: test the spear attack generator and detector on one of the group members after training it on the synthetic profiles.

Attack Generator

Harsha Dayini Akula

Build the module that turns a synthetic employee's profile into a realistic spear-phishing email, then analyze patterns across the generated attacks.

Target profile → **Attack scenario** → **Spear email** → **Pattern analysis**

What I'll own (full duration)

- Build the generator: profile → scenario → email
- Benchmark LLMs: latency, token budget, input/output limits
- Find & curate email datasets for realistic content
- Scale generation across all synthetic employees
- Vary attack type and personalization depth
- Analyze patterns across the generated attacks
- Then join the shared detection work — my attacks are the test set

Interfaces & safety

- Input: employee profiles from the profile team
- Output: labelled attack emails sent via the email server
- Produces the attack set the detection algorithm evaluates against
- Generator is modular / swappable (local or API models)
- Sandboxed: non-functional links, synthetic targets only, no real recipients

Prior Work & Repos Explored

Harsha Dayini Akula

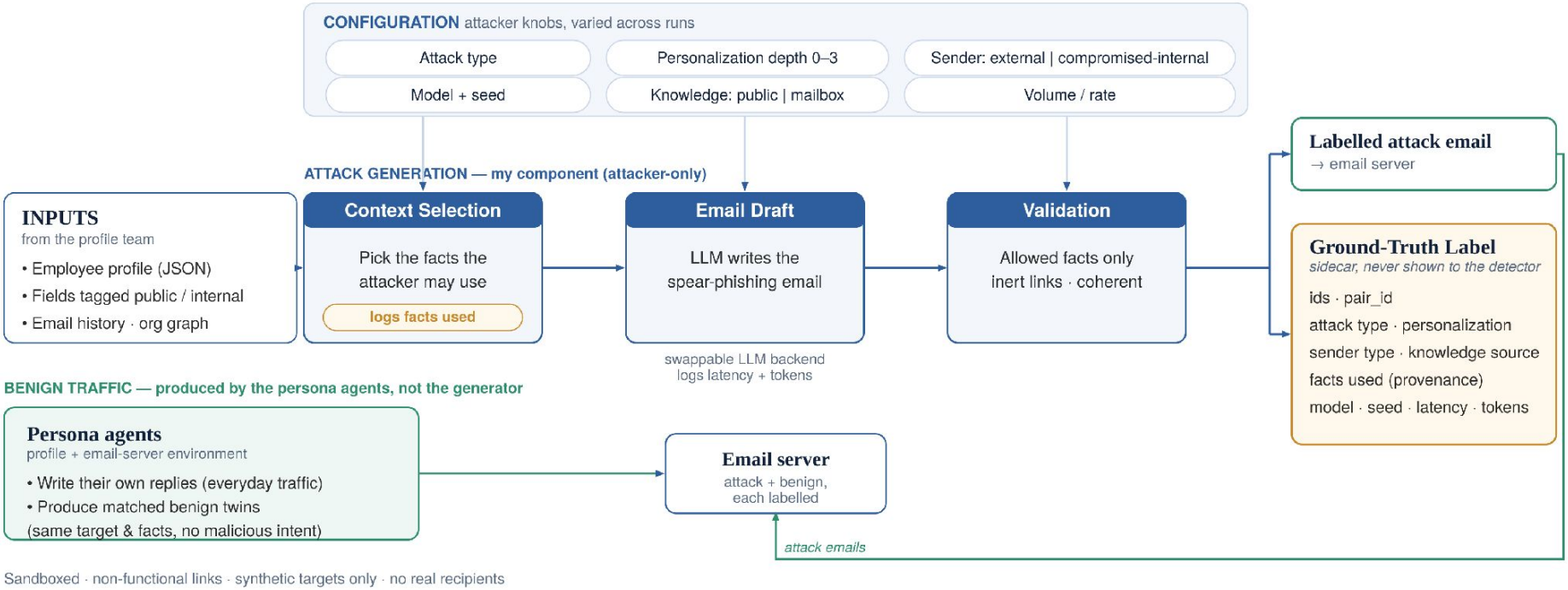
Repo / paper	What is public	Fault for GOSEE
Nahmias et al.	333 generated emails + prompts, detection code (licensed repo)	Generator itself is a proprietary system and was not released; code covers detection only
E-PhishGEN (AISec '25)	Dataset (subject, body, label, EN/IT) + generation notebook; README says "under construction"	Generic phishing: no target-profile input, no personalization controls; research notebook, license not stated
Phish-Master	Paper points to a GitHub repo	Repo holds only a one-line README
DataPhish	LLM rephrasing pipelines + email sets	Rewrites existing emails; not personalized, not spear-phishing
SpearBot · Miao et al. (NSL)	Release promised, nothing available yet	No usable code today
Vafa · Heiding · Czybik	Papers only	No code; Heiding withholds its prompts; Czybik page links no artifacts

Takeaway: no open framework turns a target profile into a labelled spear-phishing email, so we build our own.

Checked October 12, 2026 against public repos and each paper's availability statement.

Generator Architecture

Harsha Dayini Akula



Benign mode produces matched twins (same target and facts, no malicious intent) as controls for detection.

Research Questions & Experiments - Attack Generator

Harsha Dayini Akula

Research questions

Generation: can we produce realistic, controllable spear emails from synthetic profiles?

Effectiveness: does more personalization or attacker knowledge raise attack success against persona agents?

Defense: how well do existing defenses catch these attacks, and misfire on personalized benign mail?

Context: does email history improve detection, especially for compromised internal accounts?

Optional: can we detect an attacker who “knows too much” (provenance gap)?

Experiments

E1 Generator quality: realism against published sets; latency and tokens per model

E2 Personalization ladder: depth × attack type × sender type, measuring agent click / comply / report rates

E3 Detector comparison: content classifier, LLM judge, gateway baseline, persona agents; detection and false-positive rate, using matched benign twins as the hard test

E4 History ablation: same detector with and without sender–recipient history

E5 Knowledge tiers : public vs internal facts; can a detector tell the difference?

Detection

Shared work — all team members · zooms into the Detector → Evaluation stage of the system map

1 Over-knowledge

Email knows too much about you. For example, a cold outreach from someone with no prior relationship who knows your professional history.

2 Under-knowledge

A claimed relationship implies shared context that is absent. For example, following up on a meeting that never happened.

3 Channel mismatch

Knowledge sourced from one place (ex. an Instagram bio) applied through another (ex. a work email address).

PER-MESSAGE PIPELINE (RECIPIENT ENDPOINT)

Incoming email
Message + claimed sender relationship

→

Extract claims
fact + claimed relationship + implied source + requested action

→

Cross-check footprint
Recipient's Public footprint + communication history + inferred relationship graph

→

Score the gap
Over-/under-knowledge, channel mismatch

→

Route
Low gap → deliver · high gap → flag + explain signal

MEASUREMENT INSTRUMENT

Mail server / honeynet feeds real and synthetic traffic through the same steps, capturing labels for accuracy, latency, and false-positive analysis.

BASELINE COMPARISON

Existing identity/authenticity filters run in parallel as a control — expected near chance on the authentic-sender, unremarkable-content quadrant.

Detectors

Goal: flag a spear email as malicious from its content and context — even when the sender passes authentication and looks trusted.

Detectors we compare

Gateway / rule baseline: sender reputation + the current trust gap

Content classifier: fine-tuned language model on the email text

LLM-as-judge: prompt a model to rate maliciousness

History / context-aware: uses sender–recipient history + org graph

Persona agents: human-proxy recipients — do they report it?

Signals: *content · sender/auth · relationship & history · knowledge provenance (does the email know things an outsider shouldn't?)*

How we evaluate

Detection rate & false-positive rate, broken down by condition

Hard test: matched benign twins (same facts, benign intent)

Ablations: with / without history; public vs mailbox knowledge

Systems: latency & throughput in the mail path

Metrics: precision / recall / F1 · FPR · time-to-detect

Outcome signals (from sim_results): *ignore · reply · click · verify · report · comply*

Reuses the sim_results outcomes (profiles schema) and the Detector → Evaluation stage already on the system map.

Division of Labor

- Email Server - Tanmay
- Synthetic profile generation - Justin + Ayaan
- Attack generation - Harsha
- Detection - everyone (late stage once we have everything)